

Types

Overview

Python has several built-in data types used to store and organize data.

Main groups covered here:

- strings
 - lists
 - tuples
 - sets
 - dictionaries
 - numeric types
 - type conversions
-

Topic 1: String Basics

Key Vocabulary

- String literal
 - Text written directly in code using single or double quotes
 - Index
 - A character's position in a string
 - Concatenation
 - Joining strings together
-

Creating and Storing Strings

Assigning a string to a variable

```
first_name = "Armando"
```

Getting input from the user

```
first_name = input("Enter your first name")
```

or

```
first_name = input()
```

Accessing Characters by Index

You can access a character using brackets `[]`.

Example

```
first_name = "Armando"  
print(first_name[0])
```

Output:

```
A
```

Notes

- Index `0` is the first character
- Negative indexes count from the end of the string

Strings Are Immutable

Strings cannot be changed in place.

This does not work:

```
first_name[0] = "C"
```

That causes an error.

Correct approach

You must assign a new string:

```
first_name = "Elena"
```

Concatenating Strings

You can join strings using `+`.

Example

```
"New" + "York"
```

Output:

```
NewYork
```

Notes

- You can use `+` multiple times in one expression
- Concatenation only works directly with strings
- To join a number with a string, convert the number first using `str()`

Example

```
address = str(32) + " Main Street"
```

Topic 2: String Formatting

Key Vocabulary

- Formatted string literal / f-string
 - A string that includes expressions inside `{}` and evaluates them while the program runs
 - Placeholder expression
 - The expression inside `{}` that gets replaced by its value
 - Format specification
 - Extra instructions that control how a value appears
-

F-Strings

An f-string starts with `f` before the quote.

Example

```
number = 2
amount = 10.25
print(f"{number} drinks cost ${amount}")
```

Output:

```
2 drinks cost $10.25
```

Extra F-String Features

Show both the expression and the result

```
print(f"{2 * 4=}")
```

Output:

```
2 * 4=8
```

Print literal curly braces

Use double braces:

```
print(f"{{Armando Sarza}}: student")
```

Output:

```
{Armando Sarza}: student
```

Format Specification

A format specification starts with `:` inside the replacement field.

Example

```
print(f"{4:.2f}")
```

Output:

```
4.00
```

Presentation Types

Presentation types control how values are displayed.

Type	Description	Example	Output
<code>s</code>	String	<code>print(f"{name:s}")</code>	Armando
<code>d</code>	Decimal integer	<code>print(f"{number:d}")</code>	4
<code>,d</code>	Integer with commas	<code>print(f"{number:,d}")</code>	7,400

Type	Description	Example	Output
<code>0[width]d</code>	Leading zeros	<code>print(f"{number:03d}")</code>	<code>005</code>
<code>b</code>	Binary	<code>print(f"{number:b}")</code>	<code>100</code>
<code>x / X</code>	Hexadecimal	<code>print(f"{number:x}")</code>	<code>1f</code>
<code>e</code>	Exponent notation	<code>print(f"{number:e}")</code>	<code>4.400000e+01</code>
<code>f</code>	Fixed-point notation	<code>print(f"{number:f}")</code>	<code>4.000000</code>
<code>.[precision]f</code>	Fixed-point with chosen precision	<code>print(f"{number:.2f}")</code>	<code>4.00</code>
<code>,.2f</code>	Fixed-point with commas	<code>print(f"{number:,.2f}")</code>	<code>7,400.10</code>

Topic 3: List Basics

Key Vocabulary

- Container
 - A structure that groups related values together
- List
 - A mutable container created with brackets `[]`
- Sequence-type methods
 - Built-in methods for sequence types like strings and lists
- Sequence-type functions
 - Built-in functions that work on sequence types

Creating a List

```
my_list = [10, "abc"]
```

Notes

- Each item in a list is called an element
 - Lists are mutable
 - Lists are ordered by index
-

Accessing List Elements

Use brackets with an index:

```
my_list[i]
```

Updating List Elements

```
my_list[0] = "changed"
```

Adding Elements

Use `.append()` to add an item to the end.

```
my_list.append(value)
```

Removing Elements

Remove by index

```
my_list.pop(i)
```

Remove by value

```
my_list.remove(v)
```

- `pop(i)` removes the element at index `i`

- `remove(v)` removes the first matching value

Common List Functions and Methods

Function / Method	Purpose
<code>len(list)</code>	Returns the length of the list
<code>list1 + list2</code>	Concatenates two lists into a new list
<code>min(list)</code>	Returns the smallest element
<code>max(list)</code>	Returns the largest element
<code>sum(list)</code>	Returns the sum of numeric elements
<code>list.index(val)</code>	Returns the index of the first matching value
<code>list.count(val)</code>	Counts how many times a value appears

Topic 4: Tuple Basics

Key Vocabulary

- Tuple
 - A sequence type like a list, but immutable
- Named tuple
 - A simple custom data type with named attributes

Tuple Basics

A tuple stores a collection of values.

Example

```
white_house_coordinates = (38.8977, 77.0366)
```

Notes

- Tuples support `len()`, indexing, and other sequence operations
- Tuples are immutable
- Tuples are useful when values should not change
- Tuples are useful when element position matters

Example: Using a Tuple

```
white_house_coordinates = (38.8977, 77.0366)

print(f"Coordinates: {white_house_coordinates}")
print(f"Tuple length: {len(white_house_coordinates)}")
print(f"Latitude: {white_house_coordinates[0]} north")
print(f"Longitude: {white_house_coordinates[1]} west")
```

Important

This causes an error because tuples are immutable:

```
white_house_coordinates[1] = 50
```

Named Tuples

Named tuples must be imported first.

```
from collections import namedtuple
```

Creating a named tuple type

```
Car = namedtuple("Car", ["make", "model", "price", "horsepower", "seats"])
```

Creating objects

```
chevy_blazer = Car("Chevrolet", "Blazer", 32000, 275, 8)
chevy_impala = Car("Chevrolet", "Impala", 37495, 305, 5)
```

Printing them

```
print(chevy_blazer)
print(chevy_impala)
```

Notes

- Named tuple fields are accessed with dot notation
- This is easier to read than using indexes

Example:

```
chevy_blazer.price
```

Topic 5: Set Basics

Key Vocabulary

- Set
 - An unordered collection of unique elements

Properties of a Set

- Elements are unordered
- Elements are unique
- Sets do not use indexes

Creating a Set

Using `set()`

```
nums1 = set([1, 2, 3])
```

Using set literal syntax

```
nums2 = {7, 8, 9}
```

Empty set

```
empty_set = set()
```

Notes

- Do not use indexing with sets
- Sets are useful for removing duplicates

Example

```
numbers = [1, 2, 2, 3, 3, 4]  
unique_numbers = set(numbers)
```

Modifying Sets

Sets are mutable.

Function / Method	Purpose
<code>len(set)</code>	Returns the number of elements
<code>set1.update(set2)</code>	Adds elements of <code>set2</code> into <code>set1</code>
<code>set.add(value)</code>	Adds one value
<code>set.remove(value)</code>	Removes a value, raises <code>KeyError</code> if missing
<code>set.pop()</code>	Removes a random element
<code>set.clear()</code>	Removes all elements

Set Operations

Operation	Purpose
<code>set.intersection(set_a, set_b, ...)</code>	Returns elements common to all sets
<code>set.union(set_a, set_b, ...)</code>	Returns all unique elements from all sets
<code>set.difference(set_a, set_b, ...)</code>	Returns elements in the first set that are not in the others
<code>set_a.symmetric_difference(set_b)</code>	Returns elements found in exactly one of the two sets

Topic 6: Dictionary Basics

Key Vocabulary

- Dictionary
 - A Python container used for associative relationships
- Key
 - The lookup term in a dictionary
- Value
 - The data associated with a key

Dictionary Basics

A dictionary stores key-value pairs.

Example

```
players = {  
    "Lionel Messi": 10,
```

```
"Cristiano Ronaldo": 7  
}
```

Accessing Dictionary Entries

Use a key inside brackets:

```
prices = {"apples": 1.99, "oranges": 1.49}  
print(f"The price of apples is {prices['apples']}")
```

Output:

```
The price of apples is 1.99
```

Important

- Dictionaries do not use numeric indexes
- If the key does not exist, Python raises a `KeyError`

Adding, Modifying, and Removing Entries

Add a new entry

```
prices["raisins"] = 2.13
```

Modify an existing entry

```
prices["raisins"] = 2.25
```

Remove an entry

```
del prices["raisins"]
```

Notes

- Dictionaries are mutable
- Reusing an existing key replaces the old value
- Deleting a missing key raises `KeyError`

Topic 7: Common Data Types

Numeric Types

Type	Notes
<code>int</code>	Used for integers
<code>float</code>	Used for floating-point numbers

Sequence and Mapping Types

Type	Notes
<code>string</code>	Sequence type used for text
<code>list</code>	Mutable sequence with ordered elements
<code>tuple</code>	Immutable sequence with ordered elements
<code>set</code>	Mutable container with unordered unique elements
<code>dictionary</code>	Mapping type using key-value pairs

Notes About Sequence Types

- Strings, lists, and tuples are ordered by position
- Their elements can be accessed with indexes
- `list` is mutable
- `tuple` is immutable
- Lists and tuples can contain different data types

Notes About Dictionaries

- A dictionary is a container
 - Its elements do not have positional order like a list
 - Each value is associated with a key
-

Choosing a Container Type

Use a list when:

- data has an order
- items may need to change

Example:

- lines of text on a page

Use a tuple when:

- data should not change
- element position matters

Use a dictionary when:

- relationships matter more than position
- you want key-value pairs

Example:

- student names and grades

Use a set when:

- you want unique values only
 - order does not matter
-

Topic 8: Type Conversions

Key Vocabulary

- Type conversion
 - Changing one type into another
 - Implicit conversion
 - Conversion done automatically by Python
-

Notes on Implicit Conversion

Examples:

- `1 + 2` returns an integer
 - `1 + 2.0` returns a float
 - `1.0 + 2.0` returns a float
-

Common Conversion Rules

- Integer to float:
 - `25` becomes `25.0`
 - Float to integer:
 - `4.9` becomes `4`
 - Strings can be converted to numbers only if the text is in a valid numeric format
-

Conversion Functions

Function	Purpose	Can Convert
<code>int()</code>	Creates integers	integer, float, strings containing integers
<code>float()</code>	Creates floats	integer, float, strings containing integers or decimals

Function	Purpose	Can Convert
<code>str()</code>	Creates strings	almost any type

Examples

```
int(4.9)
```

Output:

```
4
```

```
float(25)
```

Output:

```
25.0
```

```
str(32)
```

Output:

```
"32"
```

Quick Summary

Strings

- ordered
- immutable
- accessed by index

Lists

- ordered
- mutable
- good for changing collections

Tuples

- ordered
- immutable
- good when values should stay fixed

Sets

- unordered
- unique values only
- no indexes

Dictionaries

- key-value pairs
- mutable
- good for relationships

Type Conversion

- `int()`
- `float()`
- `str()`